사용자 매뉴얼 (User Manual)

LLM Scheduler 설치, 설정, 운영 가이드

작성일: 2026-01-25 버전: 1.0.0

목차

1. 설치 및 설정
2. API 사용법
3. 대시보드 사용법
4. 스케줄러 운영
5. 트러블슈팅
6. FAQ

---

1. 설치 및 설정

1.1 시스템 요구사항

1.1.1 하드웨어 요구사항

구성 요소	최소	권장
CPU	2코어	4코어 이상
RAM	4GB	8GB 이상
디스크	10GB	50GB 이상 (SSD 권장)
네트워크	100Mbps	1Gbps

1.1.2 소프트웨어 요구사항

소프트웨어	최소 버전	권장 버전	필수 여부
Node.js	20.0.0	20 LTS	필수
npm	9.0.0	최신	필수
Docker	24.0.0	최신	권장
Docker Compose	2.20.0	최신	권장
Redis	7.0	7.2+	필수
MongoDB	7.0	7.0+	필수
Ollama	최신	최신	선택 (로컬 LLM)

1.2 설치 방법

1.2.1 방법 1: Docker Compose (권장)

```
# 1. 프로젝트 클론
git clone https://github.com/your-repo/llm-scheduler.git
cd llm-scheduler

# 2. 환경 변수 설정
cp .env.example .env
# .env 파일 편집

# 3. 모든 서비스 시작
docker-compose up -d
```

```
# 4. 상태 확인
docker-compose ps

# 5. 로그 확인
docker-compose logs -f
```

1.2.2 방법 2: 수동 설치

```
# 1. Redis 설치 및 시작
# macOS
brew install redis
brew services start redis

# Ubuntu/Debian
sudo apt-get install redis-server
sudo systemctl start redis

# 2. MongoDB 설치 및 시작
# macOS
brew tap mongodb/brew
brew install mongodb-community
brew services start mongodb-community

# Ubuntu/Debian
sudo apt-get install mongodb
sudo systemctl start mongodb

# 3. 애플리케이션 설치
cd llm-scheduler
npm install

# 4. 개발 서버 시작
npm run dev
```

1.2.3 Ollama 설치 (선택사항)

```
# macOS/Linux
curl https://ollama.ai/install.sh | sh

# 모델 다운로드
ollama pull llama2
ollama pull mistral

# Ollama 서버 시작 (백그라운드)
ollama serve &

# 모델 목록 확인
ollama list
```

1.3 환경 설정

1.3.1 .env 파일 설정

```
# ===== 서버 설정 =====
NODE_ENV=development
PORT=3000

# ===== Redis 설정 =====
```

```
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=
# 프로덕션에서는 반드시 비밀번호 설정

# ===== MongoDB 설정 =====
MONGODB_URI=mongodb://localhost:27017/llm-scheduler
MONGODB_DB_NAME=llm-scheduler

# ===== Ollama 설정 =====
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama2

# ===== OpenAI 설정 (선택사항) =====
OPENAI_API_KEY=sk-your-api-key-here
OPENAI_MODEL=gpt-3.5-turbo-instruct

# ===== 스케줄러 설정 =====
DEFAULT_SCHEDULER=fcfs
# 옵션: fcfs, priority, mlfq, wfq

MAX_CONCURRENT_REQUESTS=5
QUEUE_RETENTION_MS=86400000
# 86400000ms = 24시간

# ===== WebSocket 설정 =====
SOCKET_CORS_ORIGIN=http://localhost:3000

# ===== 로깅 =====
LOG_LEVEL=info
# 옵션: error, warn, info, debug
```

1.3.2 스케줄러별 추가 설정

MLFQ 설정:

```
MLFQ_QUEUE_COUNT=4
MLFQ_BOOST_INTERVAL_MS=5000
MLFQ_TIME_QUANTA=1000,3000,8000,0
# 0은 무제한을 의미
```

WFQ 설정:

```
WFQ_DEFAULT_WEIGHT=10
WFQ_ENTERPRISE_WEIGHT=100
WFQ_PREMIUM_WEIGHT=50
WFQ_STANDARD_WEIGHT=10
WFQ_FREE_WEIGHT=1
```

---

2. API 사용법

2.1 기본 API

2.1.1 헬스 체크

```
curl http://localhost:3000/api/health
```

응답:

```
{
  "status": "ok",
  "timestamp": "2026-01-25T10:30:00.000Z",
  "scheduler": {
    "type": "fcfs",
    "status": "running"
  },
  "services": {
    "redis": "connected",
    "mongodb": "connected"
  }
}
```

2.1.2 요청 제출

```
curl -X POST http://localhost:3000/api/requests \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "What is quantum computing?",
    "priority": 2,
    "maxTokens": 500,
    "userId": "user-123",
    "metadata": {
      "sessionId": "session-456",
      "source": "web"
    }
  }'
```

응답:

```
{
  "success": true,
  "requestId": "req-abc123xyz",
  "status": "queued",
  "position": 3,
  "estimatedWaitTime": 15000
}
```

2.1.3 요청 상태 조회

```
curl http://localhost:3000/api/requests/req-abc123xyz
```

응답:

```
{
  "requestId": "req-abc123xyz",
  "status": "processing",
  "prompt": "What is quantum computing?",
  "result": null,
  "error": null,
  "createdAt": "2026-01-25T10:25:00.000Z",
  "startedAt": "2026-01-25T10:28:00.000Z",
  "completedAt": null
}
```

상태 값: | 상태 | 설명 | |——|——| | pending | 아직 큐에 추가되지 않음 | | queued | 대기열에서 대기 중 | | processing | 현재 처리 중 | | completed | 처리 완료 | | failed | 처리 실패 | | cancelled | 사용자에 의해 취소됨 |

2.1.4 요청 취소

```
curl -X DELETE http://localhost:3000/api/requests/req-abc123xyz
```

응답:

```
{
  "success": true,
  "message": "Request req-abc123xyz has been cancelled"
}
```

2.2 대기열 관리

2.2.1 대기열 상태 조회

```
curl http://localhost:3000/api/queue/status
```

응답:

```
{
  "scheduler": {
    "name": "fcfs-queue",
    "type": "fcfs",
    "concurrency": 2
  },
  "queue": {
    "waiting": 15,
    "active": 2,
    "completed": 145,
    "failed": 3,
    "delayed": 0,
    "paused": false
  },
  "metrics": {
    "averageWaitTime": 12000,
    "averageProcessingTime": 5000,
    "throughput": 12.5
  }
}
```

2.2.2 대기열 목록 조회

```
curl "http://localhost:3000/api/queue/requests?limit=10&offset=0&status=queued"
```

2.3 스케줄러 관리

2.3.1 알고리즘 변경

```
curl -X POST http://localhost:3000/api/scheduler/algorithm \
  -H "Content-Type: application/json" \
  -d '{
    "algorithm": "mlfq",
    "config": {
      "queues": 4,
      "timeQuantum": 5000
    }
  }'
```

지원하는 알고리즘: | 알고리즘 | 설명 | |———-|——| | fcfs | First-Come, First-Served | | priority | 우선순위 기반 스케줄링 | | mlfq | Multi-Level Feedback Queue | | wfq | Weighted Fair Queuing |

2.3.2 스케줄러 일시정지/재개

```
# 일시정지
curl -X POST http://localhost:3000/api/scheduler/pause

# 재개
curl -X POST http://localhost:3000/api/scheduler/resume
```

2.4 메트릭 조회

```
curl http://localhost:3000/api/metrics
```

응답:

```
{
  "period": {
    "start": "2026-01-25T00:00:00.000Z",
    "end": "2026-01-25T10:30:00.000Z"
  },
  "requests": {
    "total": 200,
    "successful": 195,
    "failed": 5
  },
  "performance": {
    "averageWaitTime": 12000,
    "averageProcessingTime": 5000,
    "medianWaitTime": 10000,
    "p95WaitTime": 25000,
    "p99WaitTime": 40000
  },
  "throughput": {
    "requestsPerMinute": 12.5,
    "requestsPerHour": 750
  }
}
```

---

3. 대시보드 사용법

3.1 대시보드 접속

```
# 대시보드 서버 시작 (별도 프로세스)
npm run dashboard

# 브라우저에서 접속
open http://localhost:3001
```

3.2 대시보드 기능

3.2.1 실시간 대기열 모니터링

- 대기 중 요청 수
- 처리 중 요청 수
- 완료된 요청 수
- 실패한 요청 수

3.2.2 성능 메트릭

- 평균 대기 시간 차트
- 처리량 그래프

- 알고리즘별 성능 비교

3.2.3 요청 관리

- 요청 목록 조회
- 요청 상세 정보
- 요청 취소

3.3 WebSocket 연결

```
const io = require('socket.io-client');

const socket = io('http://localhost:3000');

// 연결 성공
socket.on('connect', () => {
  console.log('Connected to dashboard');
});

// 대기열 업데이트 구독
socket.emit('subscribe:queue');

// 대기열 업데이트 수신
socket.on('queue:updated', (data) => {
  console.log('Queue status:', data);
});

// 특정 요청 상태 구독
socket.emit('subscribe:request', { requestId: 'req-abc123' });

socket.on('request:updated', (data) => {
  console.log('Request status:', data);
});
```

---

4. 스케줄러 운영

4.1 알고리즘 선택 가이드

상황	권장 알고리즘
단순한 배치 작업	FCFS
유료/무료 사용자 구분	Priority
대화형 서비스	MLFQ
멀티테넌트 SaaS	WFQ

4.2 성능 튜닝

4.2.1 동시 처리량 조정

```
# .env
MAX_CONCURRENT_REQUESTS=10
```

가이드라인: | 시스템 사양 | 권장 동시 처리량 | |—————-|————————| | 2코어, 4GB | 2-3 | | 4코어, 8GB | 5-10 | | 8코어, 16GB | 10-20 |

4.2.2 메모리 최적화

```
# 큐 보관 기간 (밀리초)
QUEUE_RETENTION_MS=3600000  # 1시간

# Redis 메모리 설정
redis-cli CONFIG SET maxmemory 1gb
redis-cli CONFIG SET maxmemory-policy allkeys-lru
```

4.3 모니터링

4.3.1 로그 확인

```
# 실시간 로그
npm run dev

# 프로덕션 로그 (PM2)
pm2 logs llm-scheduler

# 로그 레벨 조정
LOG_LEVEL=debug npm run dev
```

4.3.2 상태 점검 스크립트

```
#!/bin/bash
# health-check.sh

# API 헬스 체크
response=$(curl -s http://localhost:3000/api/health)
status=$(echo $response | jq -r '.status')

if [ "$status" = "ok" ]; then
    echo "System is healthy"
else
    echo "System is unhealthy!"
    exit 1
fi

# 대기열 상태
queue_response=$(curl -s http://localhost:3000/api/queue/status)
waiting=$(echo $queue_response | jq -r '.queue.waiting')
echo "Waiting requests: $waiting"
```

---

5. 트러블슈팅

5.1 일반적인 문제

5.1.1 서버가 시작되지 않음

문제: 서버 시작 시 오류 발생

해결 방법:

```
# 포트 충돌 확인
lsof -i :3000

# 다른 포트 사용
PORT=3001 npm run dev
```

```
# 또는 충돌 프로세스 종료
kill -9 <PID>
```

5.1.2 Redis 연결 실패

문제: "Error: Redis connection refused"

해결 방법:

```
# Redis 상태 확인
redis-cli ping
# 응답: PONG

# Redis 시작
brew services start redis  # macOS
sudo systemctl start redis # Linux

# Docker 사용
docker-compose up -d redis
```

5.1.3 MongoDB 연결 실패

문제: "MongoServerError: Connection refused"

해결 방법:

```
# MongoDB 상태 확인
mongosh --eval "db.adminCommand('ping')"

# MongoDB 시작
brew services start mongodb-community  # macOS
sudo systemctl start mongodb           # Linux

# Docker 사용
docker-compose up -d mongodb
```

5.1.4 Ollama 연결 실패

문제: "Error: Ollama service unavailable"

해결 방법:

```
# Ollama 상태 확인
curl http://localhost:11434/api/tags

# Ollama 시작
ollama serve

# 모델 확인 및 다운로드
ollama list
ollama pull llama2
```

5.2 성능 문제

5.2.1 응답 지연

증상: 요청 처리가 느림

진단:

```
curl http://localhost:3000/api/metrics
# averageWaitTime, averageProcessingTime 확인
```

해결: 1. 동시 처리량 증가: `MAX_CONCURRENT_REQUESTS=10` 2. 더 빠른 모델 사용: `OLLAMA_MODEL=tinyllama` 3. 스케줄러 변경: `DEFAULT_SCHEDULER=mlfq`

5.2.2 메모리 부족

증상: "JavaScript heap out of memory"

해결:

```
# Node.js 메모리 증가
NODE_OPTIONS="--max-old-space-size=4096" npm run start

# 큐 보관 기간 단축
QUEUE_RETENTION_MS=3600000  # 1시간
```

5.3 로그 분석

```
# 에러 로그만 필터링
grep "error" logs/combined.log

# 특정 요청 추적
grep "req-abc123" logs/combined.log

# 최근 100줄 확인
tail -100 logs/combined.log
```

---

6. FAQ

Q1: 어떤 스케줄러를 사용해야 하나요?
A: 사용 사례에 따라 다릅니다: - 단순한 배치 작업: FCFS - 유료/무료 사용자 구분: Priority - 대화형 서비스 (빠른 응답 필요): MLFQ - 멀티테넌트 **SaaS**: WFQ

Q2: Ollama 없이 사용할 수 있나요?
A: 네, OpenAI API를 사용할 수 있습니다.

```
OPENAI_API_KEY=sk-your-key
OPENAI_MODEL=gpt-3.5-turbo-instruct
```

Q3: 프로덕션 배포는 어떻게 하나요?
A: PM2를 사용한 프로덕션 배포:

```
# 빌드
npm run build

# PM2로 시작
npm install -g pm2
pm2 start dist/index.js --name llm-scheduler
pm2 save
pm2 startup
```

Q4: 스케줄러를 런타임에 변경할 수 있나요?
A: 네, API를 통해 변경 가능합니다:

```
curl -X POST http://localhost:3000/api/scheduler/algorithm \
  -H "Content-Type: application/json" \
  -d '{"algorithm": "mlfq"}'
```

Q5: 요청 우선순위는 어떻게 설정하나요?
A: 요청 제출 시 `priority` 필드를 사용합니다:

```
{
  "prompt": "Your prompt",
  "priority": 3  // 0=LOW, 1=NORMAL, 2=HIGH, 3=URGENT
}
```

Q6: WFQ에서 테넌트 가중치는 어떻게 변경하나요?
A: API를 통해 테넌트를 등록/수정합니다:

```
curl -X POST http://localhost:3000/api/tenants \
  -H "Content-Type: application/json" \
  -d '{
    "tenantId": "new-tenant",
    "name": "New Tenant",
    "tier": "premium",
    "weight": 50
  }'
```

기술 지원

- **GitHub Issues**: 버그 리포트 및 기능 요청
- 문서: 이 매뉴얼 및 API 문서 참조
- 커뮤니티: GitHub Discussions

문서 버전: 1.0.0 최종 업데이트: 2026-01-25 작성자: LLM Scheduler 팀